FreshGuard

Language-conditioned, quality-aware robotic fruit sorting

COMPETITION	AMD Radeon Hackathon 2026
TRACK	Track 3 - Physical AI
PLATFORM	AMD Radeon GPU + ROCm + Genesis + LeRobot
AUTHOR	cairen9 - Solo developer
SOURCE	github.com/cairen9/freshguard-physical-ai
DATE	August 6, 2026

A learned visuomotor policy and a ripeness-routed force guard improve quality-adjusted throughput while preserving transparent, reproducible evidence.

1. Executive summary

FreshGuard is a simulated packing-line application in which a Franka Panda robot interprets a language instruction, selects a ripe or unripe banana, and places it in the correct quality bin. The system extends a SmolVLA policy with a class-routed low-level controller: ripe fruit retains learned finger position control, while unripe fruit uses 6 N-per-finger force control with contact-triggered, latched backoff.

99%	72%	0%	5.58 ms
Task success	Damage-free success	Wrong-bin rate	Inference latency p95

Application objective

- Maximize correctly sorted output that also remains below a frozen contact-risk threshold.
- Expose the trade-off between placement reliability, contact force, and useful throughput.
- Demonstrate end-to-end training, inference, simulation, and scaling on AMD Radeon and ROCm.

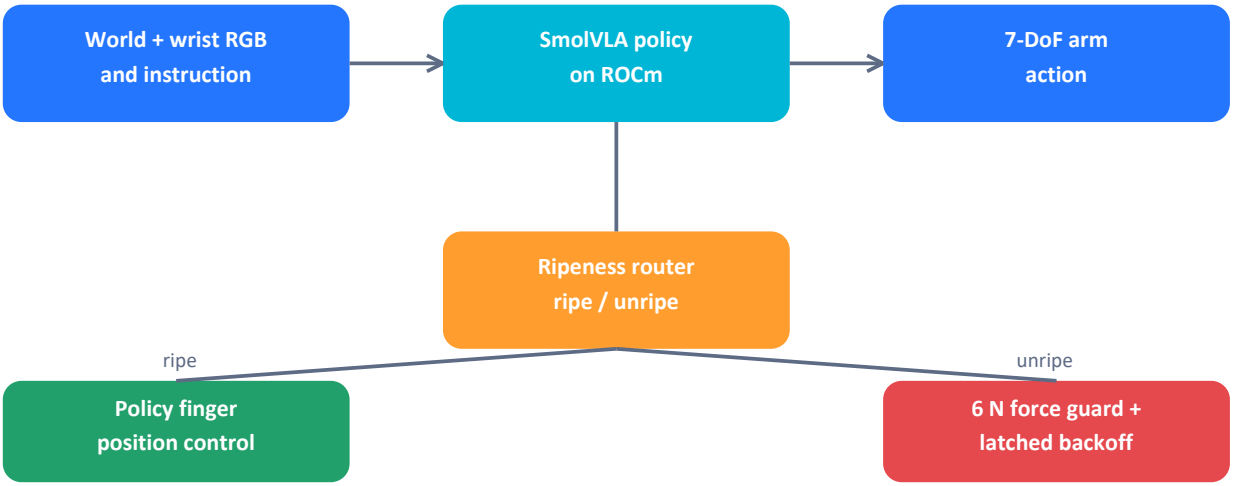
> Important: damage-free success is a simulation risk proxy. The 18 N ripe and 26 N unripe thresholds are not calibrated biological bruise limits.

Key contribution

The frozen hybrid controller raises holdout success from 84% to 99% and damage-free success from 64% to 72%. Mean peak contact rises slightly from 17.17 N to 18.18 N; therefore the contribution is honestly framed as higher quality-adjusted throughput, not universally gentler contact.

2. System architecture and solution design

End-to-end control path



Component	Role	Implementation
Task scene	Banana quality sorting with two destination bins	Genesis rigid-body simulation
Observation	External RGB, wrist RGB, robot state, instruction	LeRobot-compatible dataset
Learned policy	Target selection and 7-DoF arm action	SmolVLA inference on ROCm
Quality router	Select finger controller from ripeness instruction	Deterministic rule at evaluation
Contact guard	Finger-only trigger and latched force backoff	6 N per finger for unripe tasks
Evaluator	Success, bin accuracy, contact risk, latency	Seed-matched 100-episode holdout

Contact semantics

Contact records are resolved by Franka link index. Only left- and right-finger contact can trigger the force guard. Contact against any robot link remains part of the conservative damage-risk metric, preventing palm or arm collisions from being misclassified as gripper squeeze while still counting them as safety risks.

3. Dataset, training, and evaluation methodology

Training data

The recorder generates language-conditioned demonstrations for ripe and unripe tasks. Each episode includes two RGB streams, robot state, action targets, and the instruction. Position jitter and physics randomization broaden the distribution while retaining a controlled task definition.

Stage	Episodes / seeds	Randomization	Purpose
Training demonstrations	100 episodes	2 cm jitter; friction 0.75-1.25; mass 0.8-1.2	Train SmolVLA
Tuning	20-episode sweeps	Fixed seed ranges	Select guard forces before holdout
Final holdout	100 seeds: 8000-8099	2 cm jitter; friction 0.65-1.35; mass 0.7-1.3	Frozen comparison
Demo	6 seeds from 9000	Same task family	Show complete workflow

Fair evaluation controls

- The hybrid configuration was frozen before the final 100-seed holdout.
- Baseline and hybrid runs use identical scene seeds and policy-side RNG seeds per episode.
- No retry is enabled, so final success equals first-attempt success.
- Wrong-target and wrong-bin outcomes are reported separately from grasp failure.
- Latency and wall time are captured alongside task metrics.

Primary metrics

Metric	Definition
Task success	Requested target ends inside its expected bin.
Damage-free success	Task success with no target-to-robot contact above the class threshold.
Finger damage risk	At least one finger contact exceeds the class threshold.
Wrong-bin / wrong-target	Sorting errors reported independently from motion completion.

4. Innovation and technical contribution

Contribution	Technical detail	Why it matters
Language-routed hybrid control	The ripeness instruction selects learned or force-controlled fingers.	One policy supports class-specific handling behavior.
Latched force guard	Finger contact triggers a backoff state that remains active through the grasp phase.	Avoids unstable on/off behavior around a threshold.
Link-resolved sensing	Finger contact drives control; all robot contact drives conservative scoring.	Separates actuation feedback from safety accounting.
Quality-adjusted throughput	Counts only correctly placed, below-threshold outcomes per unit wall time.	Connects robotics metrics to usable packing output.
Reproducible matched evaluation	Frozen controller and shared holdout seeds with raw JSON evidence.	Supports paired claims instead of anecdotal demos.

Controller behavior

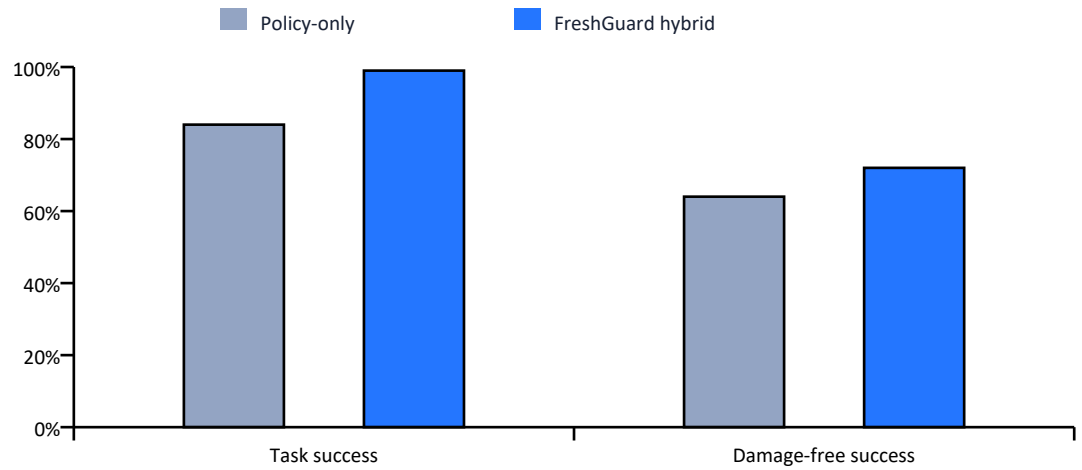
Ripe tasks keep the policy's original position-controlled fingers. Unripe tasks use force control at 6 N per finger. The contact-triggered latch backs off the force target after contact evidence, while the arm trajectory remains policy controlled. This minimal intervention preserves the learned manipulation behavior and changes only the subsystem associated with gripping.

Design boundaries

- No claim is made that simulation contact thresholds predict real bruising without calibration.
- The controller does not lower mean contact across all attempts; it increases useful below-threshold output.
- The six demo episodes illustrate operation but are not used as the formal performance claim.

5. Final results

Untouched 100-seed holdout



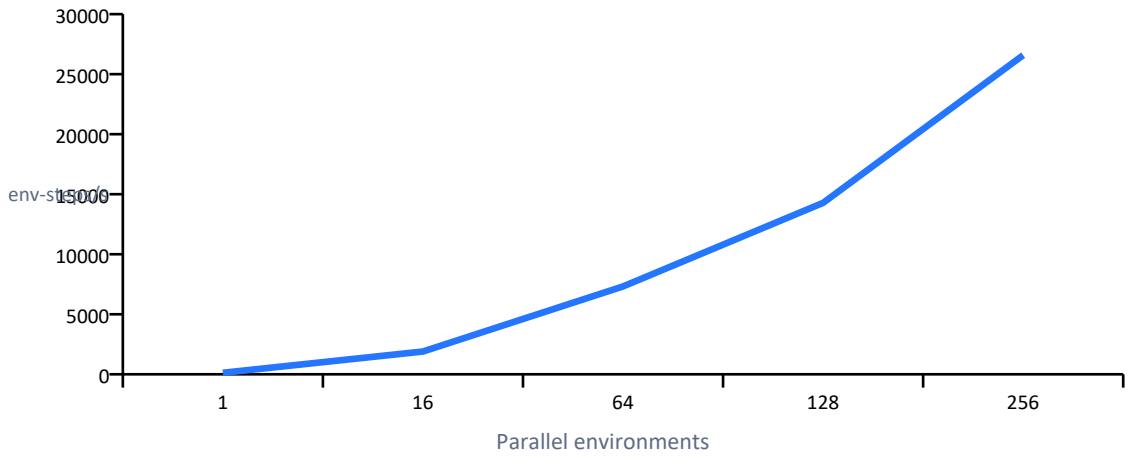
Metric	Policy-only	FreshGuard	Change
Task success	84%	99%	+15 pp
Damage-free success	64%	72%	+8 pp
Expected-bin rate	84%	99%	+15 pp
Wrong-bin rate	0%	0%	0 pp
Mean peak contact	17.17 N	18.18 N	+1.01 N
Mean episode wall time	13.67 s	12.17 s	-1.50 s
Inference latency p95	5.32 ms	5.58 ms	+0.26 ms

Using measured episode times, damage-free successful episodes per wall-second improve from approximately 0.0468 to 0.0592, a 26% increase. The gain comes primarily from the unripe route: success rises from 72% to 100% and damage-free success from 54% to 68%.

6. AMD Radeon and ROCm utilization

The frozen runs use Genesis 1.2.3 with the gs.amdgpu backend on a gfx1100 Radeon device reporting 47.98 GB of device memory. Genesis simulation and SmolVLA inference run in the same ROCm environment.

Parallel simulation scaling



Environments	env-steps/s	Speedup	Efficiency
1	124.5	1.00x	100.0%
16	1,896.4	15.23x	95.2%
64	7,316.6	58.77x	91.8%
128	14,279.1	114.69x	89.6%
256	26,579.9	213.49x	83.4%

The benchmark includes 200 warm-up steps and 1,000 steady-state measured steps. Scene construction and kernel compilation are excluded. The 256-environment result demonstrates 213.49x scaling with 83.4% parallel efficiency, supporting high-throughput synthetic data generation and evaluation.

7. Reproducibility instructions

Repository

https://github.com/cairen9/freshguard-physical-ai

Environment and training

Use the repository's uv-managed environment on the provided Radeon/ROCm instance. The complete dependency lockfile and project configuration are committed with the source.

```
uv run python franka_fruit_pick/freshguard_record.py --episodes 100 --repo-id freshguard/banana_quality_sort
--root datasets/banana_quality_sort --jitter 0.02 --dr-friction 0.75 1.25 --dr-mass 0.8 1.2
```

```
uv run python franka_fruit_pick/train_policy.py smolvla --repo-id freshguard/banana_quality_sort
--dataset-root datasets/banana_quality_sort --name freshguard_4000 --steps 4000 --save-freq 2000
```

Matched baseline

```
uv run python franka_fruit_pick/freshguard_eval.py --policy-path
outputs/train/freshguard_4000/checkpoints/last/pretrained_model --repo-id freshguard/banana_quality_sort
--dataset-root datasets/banana_quality_sort --episodes 100 --seed 8000 --grip-guard-mode off --results-out
outputs/freshguard/holdout_policy_only_seed8000.json
```

Frozen hybrid

```
uv run python franka_fruit_pick/freshguard_eval.py --policy-path
outputs/train/freshguard_4000/checkpoints/last/pretrained_model --repo-id freshguard/banana_quality_sort
--dataset-root datasets/banana_quality_sort --episodes 100 --seed 8000 --grip-guard-mode unripe-only
--unripe-grip-force-n 6 --results-out outputs/freshguard/holdout_hybrid_seed8000.json
```

Radeon scaling

```
uv run python franka_fruit_pick/freshguard_benchmark.py --n-envs 1 16 64 128 256 --warmup-steps 200 --steps
1000 --out outputs/freshguard/rocm_scaling.json
```

Compact raw JSON results, rocminfo, rocm-smi, package versions, and the evaluated Git revision should be retained under docs/freshguard_results/. Large checkpoints and datasets should be published through Git LFS, a release, or a model/dataset host.

8. Deliverables, limitations, and team

Final deliverables

Deliverable	Location / form
Source code	github.com/cairen9/freshguard-physical-ai
Technical report	FreshGuard_Technical_Report.pdf
Reproduction guide	README.md and FRESHGUARD_SUBMISSION.md
Raw evidence	docs/freshguard_results/ JSON and hardware reports
Demo	3-5 minute public or unlisted video link in the competition PR

Limitations and next steps

- Real fruit, tactile sensing, and measured bruise outcomes are required to calibrate contact thresholds.
- GPU rigid-body simulation and camera rendering do not eliminate the sim-to-real gap.
- Visual ripeness currently uses controlled material/color differences; future work should cover more varieties and real cameras.
- A container image and hosted immutable checkpoint would further reduce reproduction time.

Team member and contribution

cairen9	Solo developer: application design, FreshGuard controller implementation, data collection pipeline, policy training, paired evaluation, AMD Radeon benchmarking, documentation, and demonstration production.

Conclusion

FreshGuard demonstrates a practical Physical AI pattern: retain a learned policy for high-level visuomotor behavior and add a small, interpretable controller at the contact-sensitive interface. On the untouched holdout, the result is 99% sorting success, 72% damage-free success, zero wrong-bin placements, and measurable Radeon scaling to 256 parallel environments.